

Practical

Encoding and Embedding

One Hot Encoding (word presence) → "Is the word present or not?"

→ -0&1 ← Data

Bag of Words (word count) → "How many times does the word appear?"

← NLP

TF-IDF (count + importance weighting) → "How important is the word in this document?"

→ It reduces the weight of common words and increases the weight of more informative words

Embeddings (semantic + contextual understanding) → "What is the meaning of the text in context (word, sentence, paragraph, corpus)?"

word2vec



transformer-based model

One Hot Encoding

D1 → people watch movie
D2 → people watch cricket
D3 → people like movie
D4 → people like cricket

Document & sentence

Collection of numbers

Data

OHE vector

5

unique words from the data

(Vocabulary) {people, watch, like, movie, cricket}

Step 1:

	people	watch	like	movie	cricket
D1 →	1	1	0	1	0
D2 →	1	1	0	0	1
D3 →	1	0	1	1	0
D4 →	1	0	1	0	1

Features
(Dim)

Dimension of vector

5 ← vector Dim 5

10 ← vector Dim 10

1000 ← vector Dim 1000

Document-level representation

D1 → people watch movie → [1, 1, 0, 1, 0]

D2 → people watch cricket → [1, 1, 0, 0, 1]

D3 → people like movie → [1, 0, 1, 1, 0]

D4 → people like cricket → [1, 0, 1, 0, 1]

OHE

OHE

Data

= Logistic & naive bayes

text classification

Word-level (sequence) representation

D1 → people watch movie

[[1 0 0 0 0]

[0 1 0 0 0]

[0 0 0 1 0]]

D2 → people watch cricket

[[1 0 0 0 0]

[0 1 0 0 0]

[0 0 0 0 1]]

D3 → people like movie

[[1 0 0 0 0]

[0 0 1 0 0]

[0 0 0 1 0]]

D4 → people like cricket

[[1 0 0 0 0]

[0 0 1 0 0]

[0 0 0 0 1]]

OHE

Data

Dh

NLPE

RNN / LSTM / GRU / EBD / Attention

Recurrent Neural Network

DL

Word-level (sequence) representation

D1 → people watch movie

[[1 0 0 0 0]
[0 1 0 0 0]
[0 0 0 1 0]]

[1, 1, 0, 1, 0]

Vocabulary : {people, watch, like, movie, cricket}

D2 → people watch cricket

[[1 0 0 0 0]
[0 1 0 0 0]
[0 0 0 0 1]]

D3 → people like movie

[[1 0 0 0 0]
[0 0 1 0 0]
[0 0 0 1 0]]

D4 → people like cricket

[[1 0 0 0 0]
[0 0 1 0 0]
[0 0 0 0 1]]

→ { [1 0 0 0 0]
[0 1 0 0 0]
[0 0 0 1 0] }

Row X Col
↑ ↑
3 5

3X5

{people watch like movie cricket} ← vocab

sent

People watch cricket- watch

1X5

→ [1 1 0 0 1] ← ML

D1 → [[1 0 0 0 0]
[0 1 0 0 0]

← DL

[0 0 0 0 1]]

3X5

→ People watch cricket- people ← vocab {people, watch, cricket}

one [1, 1, 1]

clear y/n

bow [2, 1, 1]

Pros

Easy to implement
Simple binary representation
No training required
Direct mapping

0 & 1

Ask Interview

Cons

Sparse Matrix Mostly zeros → memory waste

Sparse matrix means huge vector size but very little useful information."

Vocabulary size = 10,000 words

D1 → people watch movie

[1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ... up to 10,000]

9997 value zero

High Dimensionality Vocabulary increase → vector size increase =

No Semantic Understanding Similar words also seems unrelated

OOV Problem could not handle new words

IMP

Impact

50000 12

0 & 1

ONE

D5 → people enjoy movie

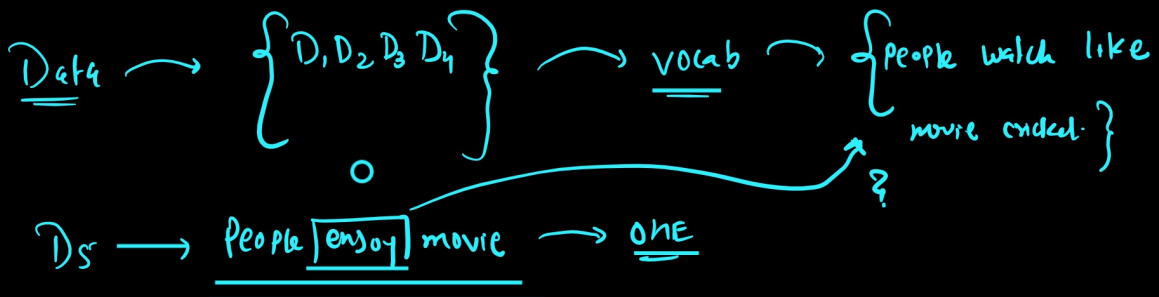
people (known)

movie (known)

enjoy (NOT in vocabulary)

enjoy ← vocab

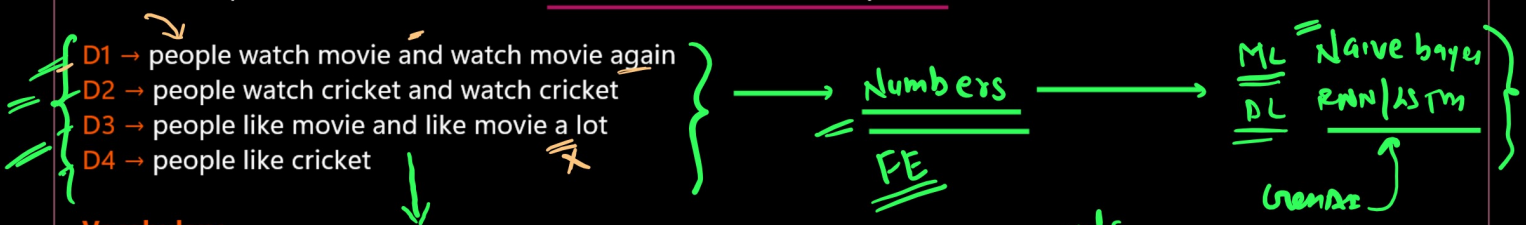
OOV



AI ← Fundamental

Bag of Words: Bag of Words captures how frequently a word is used in a meaningful context.
 "If a word repeats more in a sentence, BoW assumes it is more important."

+ exp. claim. Prediction



Vocabulary

['again', 'and', 'cricket', 'like', 'lot', 'movie', 'people', 'watch']

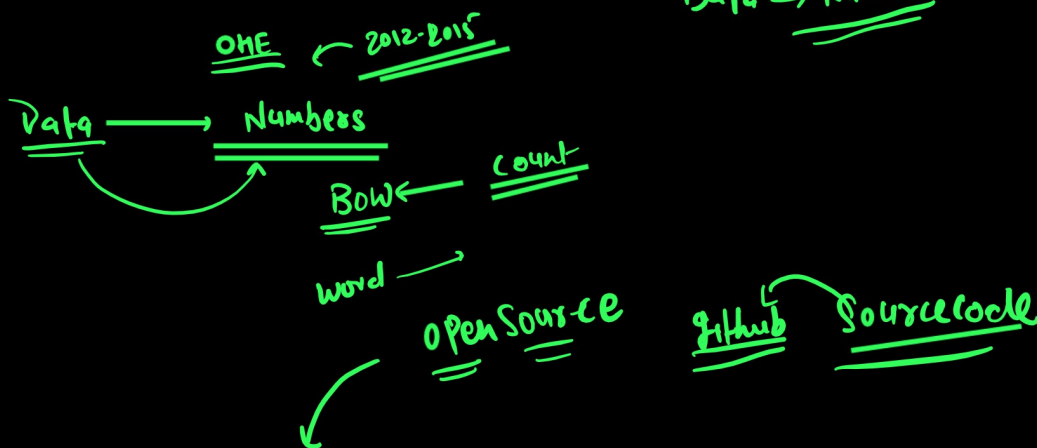
	again	and	cricket	like	lot	movie	people	watch
D1 →	1	1	0	0	0	2	1	2
D2 →	0	1	2	0	0	0	1	2
D3 →	0	1	0	2	1	2	1	0
D4 →	0	0	1	1	0	0	1	0

Features **8**
 Vector = ?
 Dimension → **8**
 8 words
 text
 vector

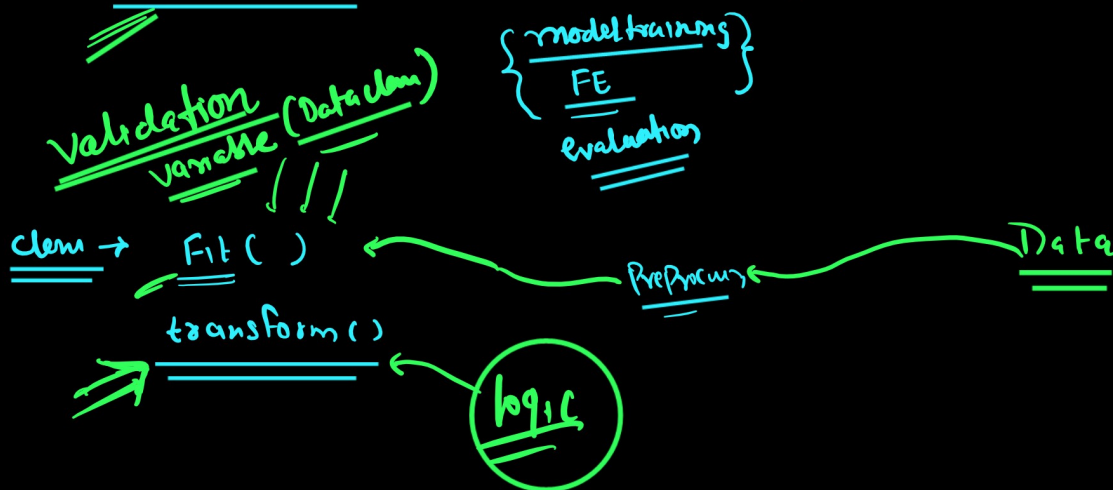
D₁ → [1 1 0 0 0 2 1 2] 1x8
 D₂ → [0 1 2 0 0 0 1 2] 1x8

Bag of Words is simple and powerful for counting words, but it fails to understand meaning, order, and context.

Data → Numbers (Context)



Sklearn → Scikit Learn ← Machine Learning



Abstract → Someone written that code

Pros

1. Easy to implement
Very simple logic
Just count word occurrences
No complex math required

2. Captures word frequency
Shows how often a word appears
More occurrences → higher importance

3. Works well before embedding for classical NLP tasks

Useful for:

- Text classification ✓
- Spam detection
- Sentiment analysis (basic level)

4. No training required
No learning phase
Direct conversion from text → numbers

BOW
Agree?

occurrence
[AI is a future AI] is imp

2014-15-16

classical NLP task
ML → NB
DL → RNN LSTM

APP

Cons

1. Ignores word order

Sentence meaning can change, but BoW won't notice

Example:

"dog bites man"

"man bites dog"

Same BoW representation

BoW vector

{Dog, bites, man}

[1 1 1] } Any diff

2. No semantic understanding

Similar words are treated completely different

Example:

"like" vs "love"

No relation captured

3. High dimensionality

Vocabulary size = vector size

Large dataset → huge vectors

vocab → 50000
↓
50000x

4. Sparse representation

Mostly values are 0

Memory inefficient

Computation waste

BoW ← Sparse

Data → Vocab ← unique

5. OOV (Out of Vocabulary) problem

New words cannot be handled

If word not in vocabulary → ignored or error

OOV
Data → Vocab ← unique
Bites, Dog, man
sent → lion Bites man
Bow → [0 1 1]
OOV

6. Overemphasizes frequent words

Repeated words may dominate

Example:

"movie movie movie"

Gets high importance even if not meaningful

Poseme

OHE → 0 & 1

BoW → Count
(freq)

= { TF-IDF → Count + Importance }

Data → vector (Number)
TF-IDF → Google
Interviews → Before NN → TF-IDF → embedding → Input → transformer

TF-IDF: Term Frequency (TF) Inverse Document Frequency (IDF)

Assignment - (Real)

- D1 → people watch cricket
- D2 → cricket watch cricket
- D3 → people give comment
- D4 → cricket give comment

Dummy Data → could be anything

Vocabulary: ['comment' 'cricket' 'give' 'people' 'watch']

(words)
unique token in data
set of words
{ ONE word2vec
bow
tf-idf }

termfreq
easy

$$TF(\text{word}, D) = \frac{(\text{occurrences of the word in the sentence})}{(\text{Total words in the given sentence})}$$

→ my name is Sunny
Sunny teach AI

$$\frac{2}{7} \leftarrow \underline{\underline{Sunny}}$$

$$IDF(\text{word}) = \log \left(\frac{\text{Total number of sentence}}{\text{Number of sentence containing the word}} \right)$$

↳ inverse document-
freq.

$$\log \left(\frac{D}{W} \right)$$

(love)

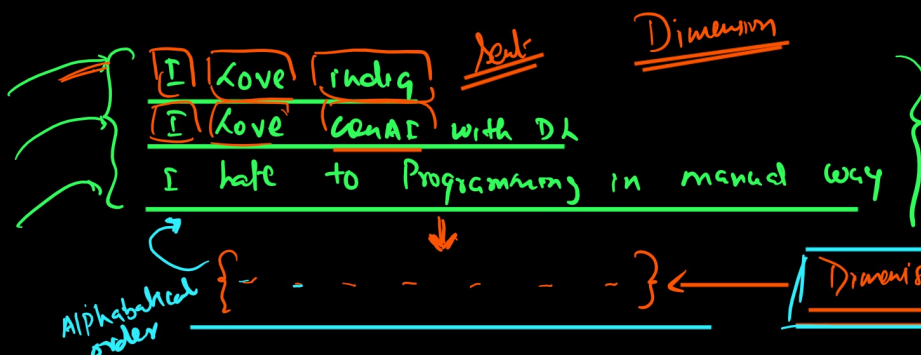
$$\left| \log \left(\frac{3}{2} \right) \right|$$

- ① I love ML
- ② I love DL
- ③ I like GenAI

TF&IDF
table → vector

TF-IDF, ONE, word2vec

Data → text ← unstructure data



D1 → people watch cricket
 D2 → cricket watch cricket
 D3 → people give comment
 D4 → cricket give comment

Vocab → {comment, cricket, give, people, watch}

TF

→	comment	<u>cricket</u>	<u>give</u>	<u>people</u>	<u>watch</u>	
		<u>TF</u> <u>IDF</u>		<u>TF</u> <u>IDF</u>	<u>TF</u> <u>IDF</u>	
D1 →	0	$(1/3) * \log(4/3)$	0	$(1/3) * \log(4/2)$	$(1/3) * \log(4/2)$	←
D2 →	0	$(2/3) * \log(4/3)$	0	0	$(1/3) * \log(4/2)$	←
D3 →	$(1/3) * \log(4/2)$	0	$(1/3) * \log(4/2)$	$(1/3) * \log(4/2)$	0	←
D4 →	$(1/3) * \log(4/2)$	$(1/3) * \log(4/3)$	$(1/3) * \log(4/2)$	0	0	←

TF-IDF

D1 → people watch cricket
 D2 → cricket watch cricket
 D3 → people give comment
 D4 → cricket give comment

$\log(4/3) \approx 0.28768$
 $\log(4/2) = \log(2) \approx 0.69315$

TF-IDF

	comment	cricket	give	people	watch
D1 →	0	0.096	0	0.231	0.231
D2 →	0	0.191	0	0	0.231
D3 →	0.231	0	0.231	0.231	0
D4 →	0.231	0.096	0.231	0	0

D1 →
 [0 0.096 0 0.231 0.231]
 [0 0.191 0 0 0.231]
 [0.231 0 0.231 0.231 0]
 [0.231 0.096 0.231 0 0]

Why TF (Term Frequency)?

Because:

More occurrences = more importance in that document

Why IDF (Inverse Document Frequency)?

Because: Common words are not useful

Why multiply TF × IDF?

Because:

TF → importance in document

IDF → importance in corpus

Why log in IDF?

Without log:

Rare words get extremely high values

Log:

Smooths values

Keeps balance

Log reduces extreme differences while keeping the importance order intact.

Log compresses large values so rare words don't overpower everything.

normalize
why → Compress

D → AI is imp AI is future AI is Deng.
TF-IDF

Vocab → {AI is imp future Deng}

Document-
AI is imp future Deng.
3 1 1 1
9 3
0.333

extreme 0 50 500 1000

log →
log(0) = [0 - 10000]
log(10000)

9.21 [0 - 9.21]

(10000)
log₂ → 13.29
log₁₀ → 4
log_e → 9.21 ← natural log
ln

Without log

$$IDF = N / DF$$

Assume:

Total documents (N) = 1000

Case 1: Rare word

Appears in 1 document

$$IDF = 1000 / 1 = 1000$$

Case 2: Medium word

Appears in 10 documents

$$IDF = 1000 / 10 = 100$$

Case 3: Common word

Appears in 500 documents

$$IDF = 1000 / 500 = 2$$

Problem

Values are too extreme:

1000 vs 100 vs 2 range

Rare word dominates everything

Model becomes unstable

With Log

Formula:

$$IDF = \log(N / DF)$$

Same cases:

Rare word:

$$\log(1000) \approx 6.9$$

Medium word:

$$\log(100) \approx 4.6$$

Common word:

$$\log(2) \approx 0.69$$

Now values become:

6.9 vs 4.6 vs 0.69

Sunny

$$IDF = \frac{\text{total no. of Doc}}{\text{in how many doc that word is doc}}$$

Sunny

D_1
 D_2
 D_3
 D_4
 $\vdots$
 D_{1000}

Sunny

$= \frac{1000}{1} = \frac{1000}{1}$

extreme

$= \log$

Common 2, 100, 1000

$$\frac{[2-1000]}{\text{normalize}}$$

$$\{\log\}$$

$$\frac{TF}{IDF}$$

log

→ final vector

$TF \times IDF$

↑ word imp in the given doc

↓ checking word imp in entire doc.

Pros

1. Captures importance of words
Not just count, but importance
Rare words → higher weight
Common words → lower weight

2. Reduces impact of common words
IDF reduces their weight

3. Better than BoW for relevance

Helps in:

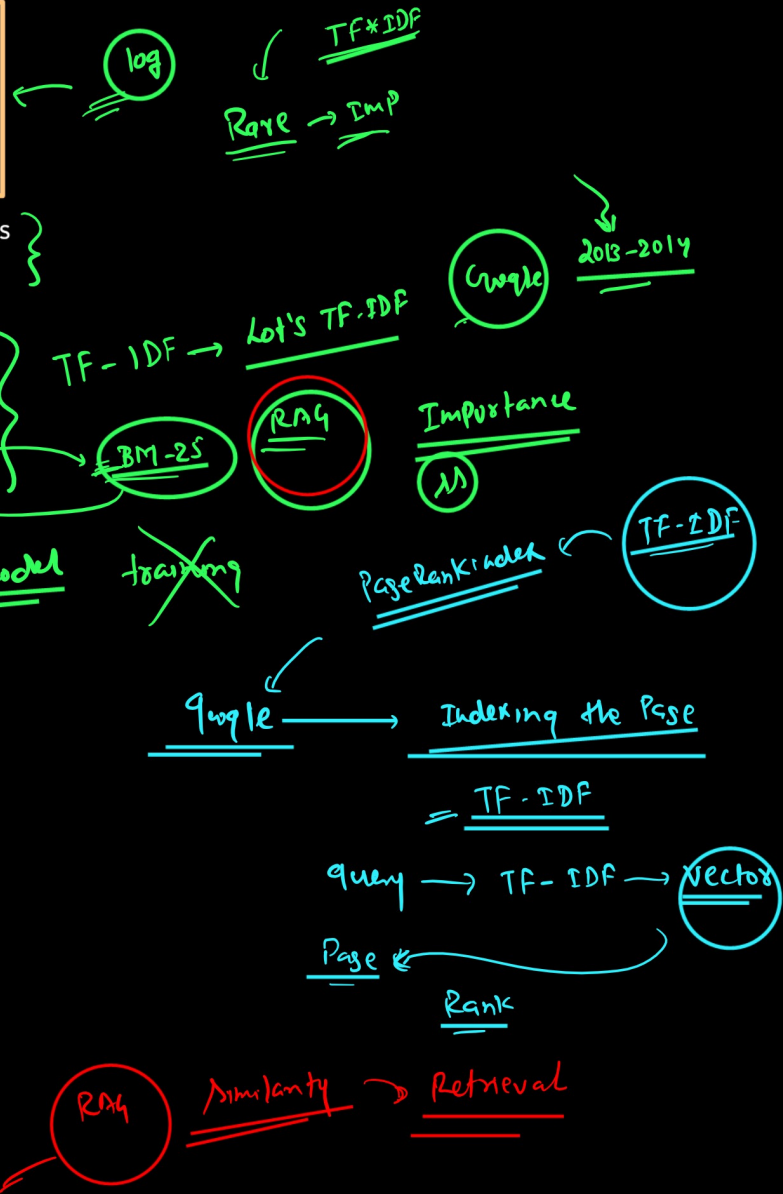
Search engines
Information retrieval
Ranking documents

4. No training required

Same as BoW
Direct calculation
No model learning needed

5. Simple and interpretable

Easy to understand mathematically
TF × IDF → clear logic



Cons

1. Ignores word order
Same problem as BoW

Example:

"dog bites man"

"man bites dog"

Same representation

2. No semantic understanding
Cannot understand meaning

Example:

"car" vs "automobile"

Treated as completely different

3. Still sparse and high dimensional

Vocabulary size = vector size

Many zeros → inefficient

4. Doesn't handle context

Same word, different meaning → same vector

Example:

"bank" (river vs finance)

5. OOV problem

New words not in vocabulary → ignored

notable Preserve

Concat

2014 → embedding

2014

TF-IDF

{ love
like }

vocab

semantic meaning of the data

huge

vocab

(10000, 50000, 100000)

parse

zeros

bank →

handle

~~training~~

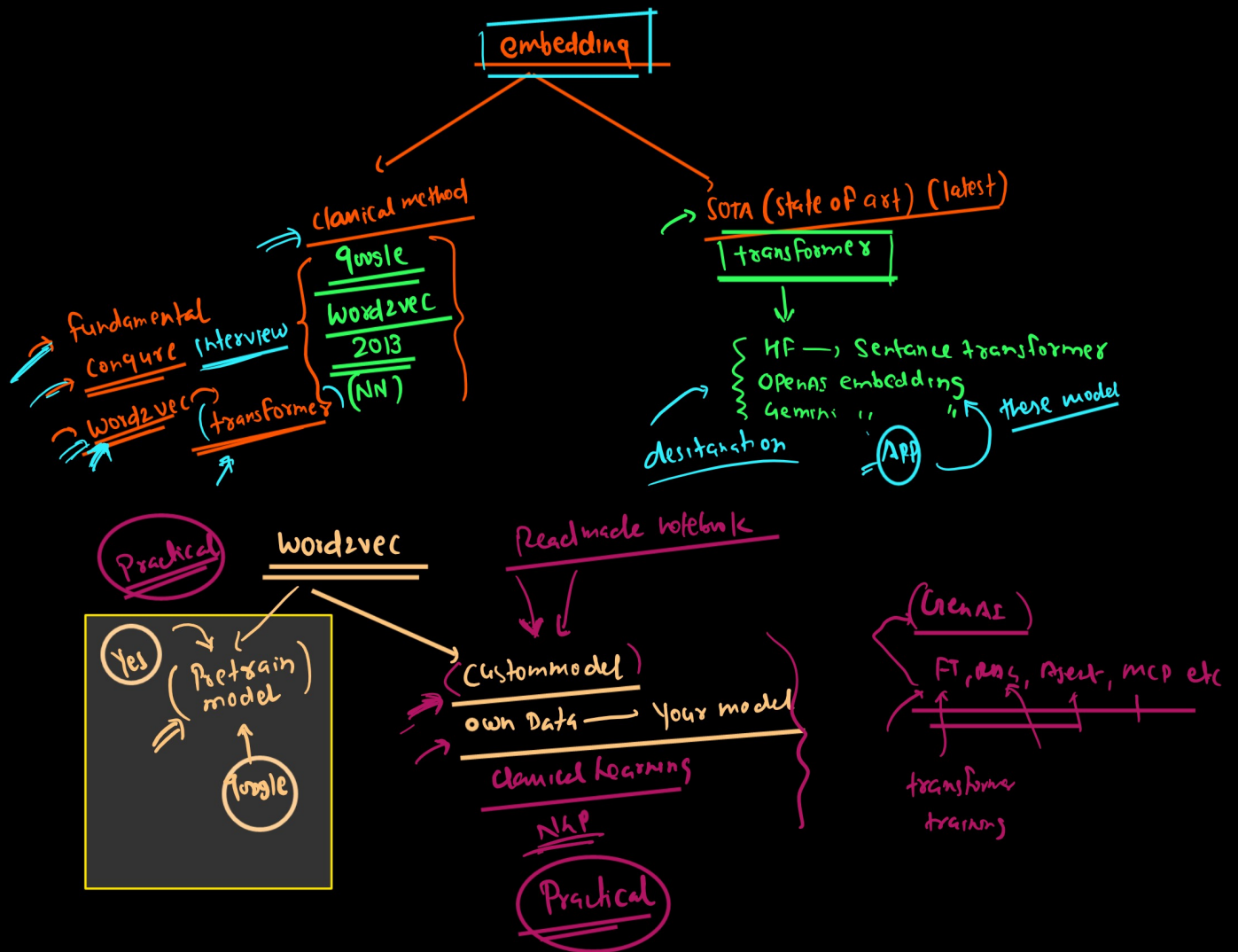
OHE, BoW, TF-IDF, BM-25 → encoding

vector

(NN, transformer) → embedding

↑
(semantic meaning)

TF-IDF improves by adding importance but still fails to understand meaning and context.



Sentence 1:

"I like this movie"

Sentence 2:

"I love this film"

Context

What happens:

OHE / BoW / TF-IDF

"like" $\neq$ "love"

"movie" $\neq$ "film"

embeddings

Context

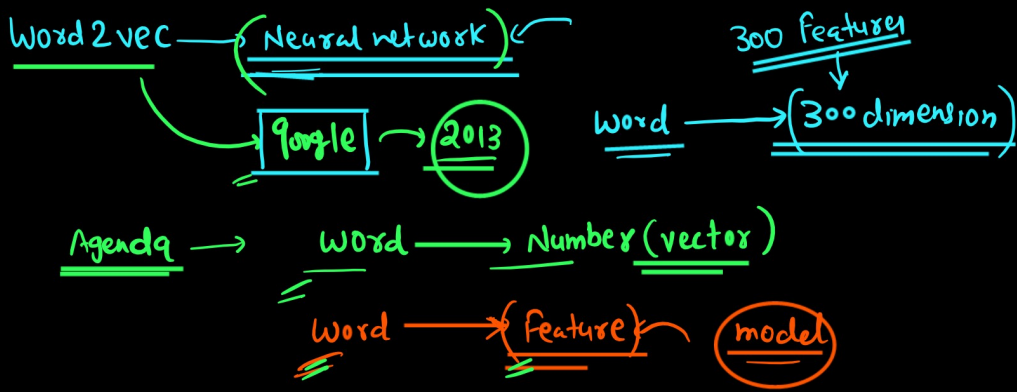
Semantic

Embeddings

"like" $\approx$ "love"

"movie" $\approx$ "film"

→ Model understands similarity

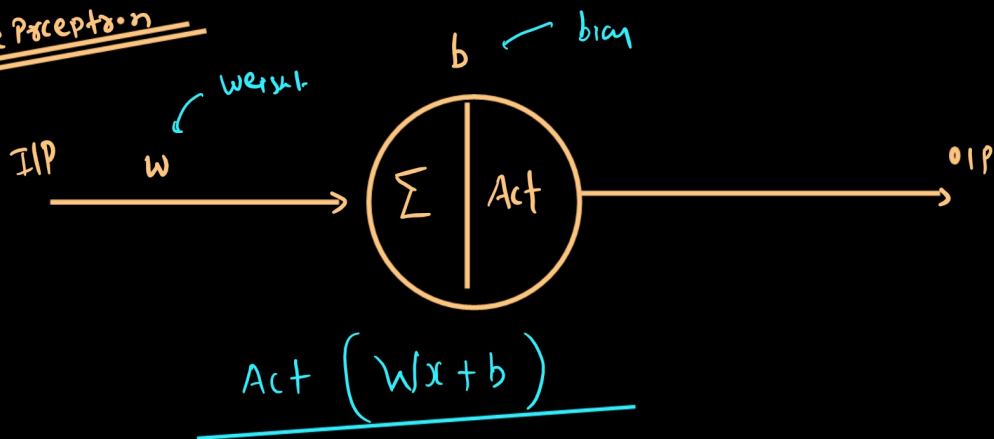


Home buy Prediction

F_1 (Size)	F_2 (Location)	F_3 (bedroom)	O/P (Y/N)
1250 Sq Ft	City center	3	Y
1140 Ft	Outskirt	2	N

← binary classification

Single Perceptron

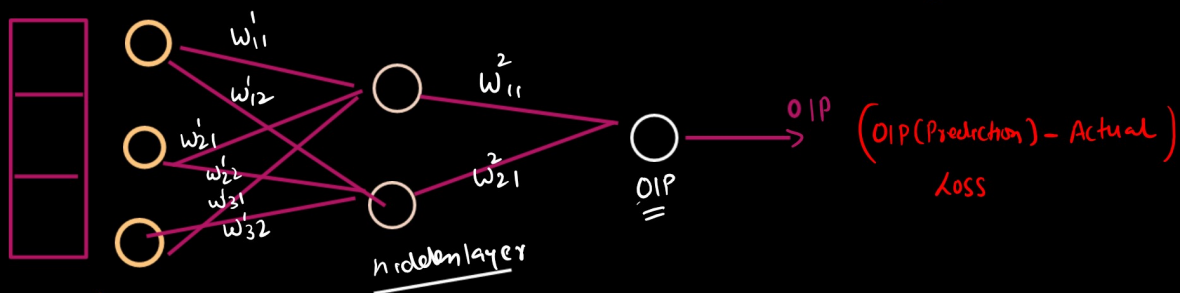


NN

- ① Input
- ② Hidden
- ③ O/P

multilayer Perceptron → multiple hidden layer

Forward Propagation $\rightarrow$ FP $\rightarrow (Wx+b)_{Act}$



BP $\rightarrow$ Optimizer $\rightarrow$ GD $\rightarrow$ Adjust the weight

backward propagation $\rightarrow$ FP+BP $\rightarrow$ 1 Epoch

Weight $\rightarrow$ init $\rightarrow$ Random weight

Learning $\rightarrow$ epoch $\rightarrow$ Loss!!!! $\rightarrow$ Best Possible weight

Word $\rightarrow$ vector

word2vec $\rightarrow$ word $\rightarrow$ feature

Analogy

word2vec

manual

teaching

model (NN)

[0-1]

KING

SPEAK

RUN

Rich

TEACHER

Sunny
TIGER

0

1

0.6

0.3

1

0

0

0.9

0

0

vector

Sunny $\rightarrow [0.1 \ 0.6 \ 0.3 \ 1]_{1 \times 5}$

5-dim

Google $\rightarrow$ 300dim

$\rightarrow$ Feature

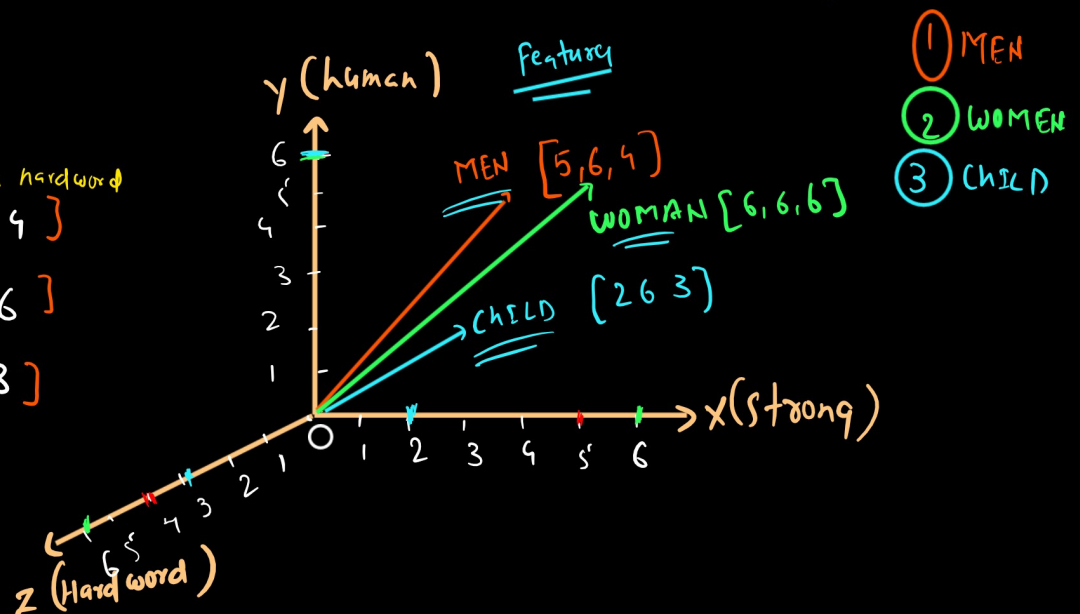
300

Visualization

men $\rightarrow$ Strong human hardword $\rightarrow [5 \ 6 \ 4]$

women $\rightarrow [6 \ 6 \ 6]$

child $\rightarrow [2 \ 6 \ 3]$



[0-1]

	Gender	Wealth	Power	Weight	Speak
KING →	1	1	0.99	0.7	1
QUEEN →	1	0.8	0.7	0.8	1
MAN →	1	0.5	0.4	0.7	1
WOMAN →	1	0.4	0.3	0.8	1
monkey →	1	0	0	0.1	0

manuel

$$\text{KING} \rightarrow [1 \ 1 \ 0.99 \ 0.7 \ 1]$$

$$\text{MAN} \rightarrow [1 \ 0.5 \ 0.4 \ 0.7 \ 1]$$

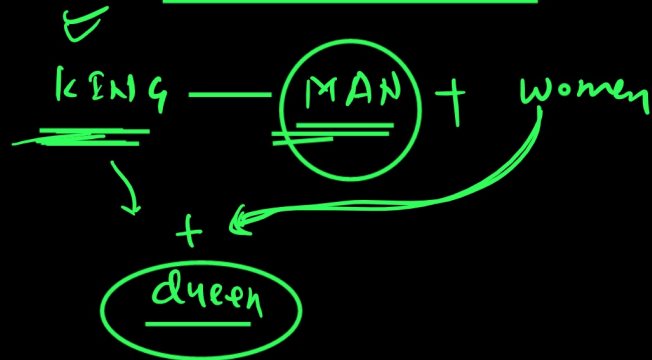
$$\text{WOMAN} \rightarrow [1 \ 0.4 \ 0.3 \ 0.8 \ 1]$$

word2vec

$$\text{Queen} = \text{KING} - \text{MAN} + \text{WOMAN}$$

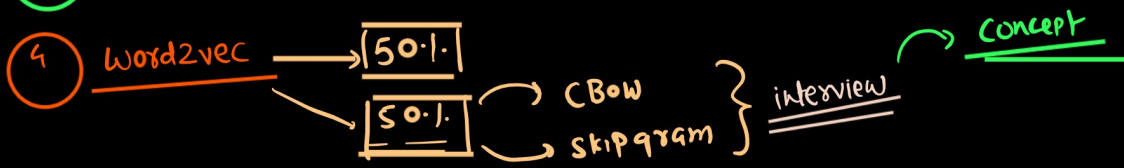
$$\text{Queen} = \begin{bmatrix} 1 \\ 0.99 \\ 0.7 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 \\ 0.5 \\ 0.4 \\ 0.7 \end{bmatrix} + \begin{bmatrix} 1 \\ 0.4 \\ 0.3 \\ 0.8 \end{bmatrix} = \begin{bmatrix} \quad \\ \quad \\ \quad \\ \quad \end{bmatrix}$$

Word embedding → vector operation

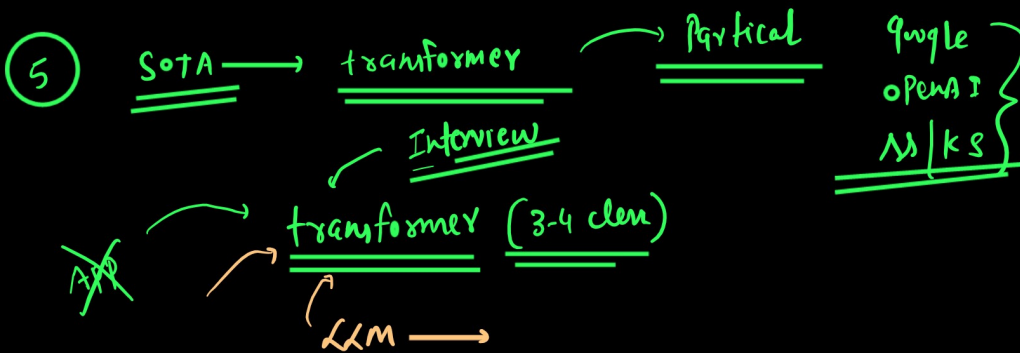


models (DL) (NN)

- 1 ONE
- 2 BOW
- 3 TF-IDF



Practical
custom training
 train your own word2vec



LLM → LLMg (text to text)
 → MMLLM → Img to text text to img

FT HF/Unloth 8-10 2A4 / 321 / mcp
AdmOPS

Vocab

I love AI and I love coding

(Vocab → uniq words (Set of words))

≈

{ "I" "love" "AI" "and" "coding" }

Sunny love AI sunny love coding → case sensitive

cap
Vocab → uniq words { "Sunny" "love" "AI" "sunny" "coding" }

- Apple ≈
- apple ≈

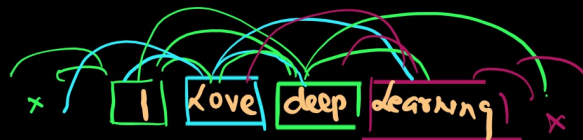
how many neighbours we going to consider surround the word.

Window Size

Data | text



Window Size = 1



<u>I/P</u> Content	<u>O/P</u> Target
Love	I
I, deep	Love
Love, Learning	deep
deep	Learning

<u>I/P</u> CONTEXT	<u>O/P</u> TARGET
Love, deep	I
I, deep, Learning	Love
I, Love, Learning	deep
Love, deep	Learning

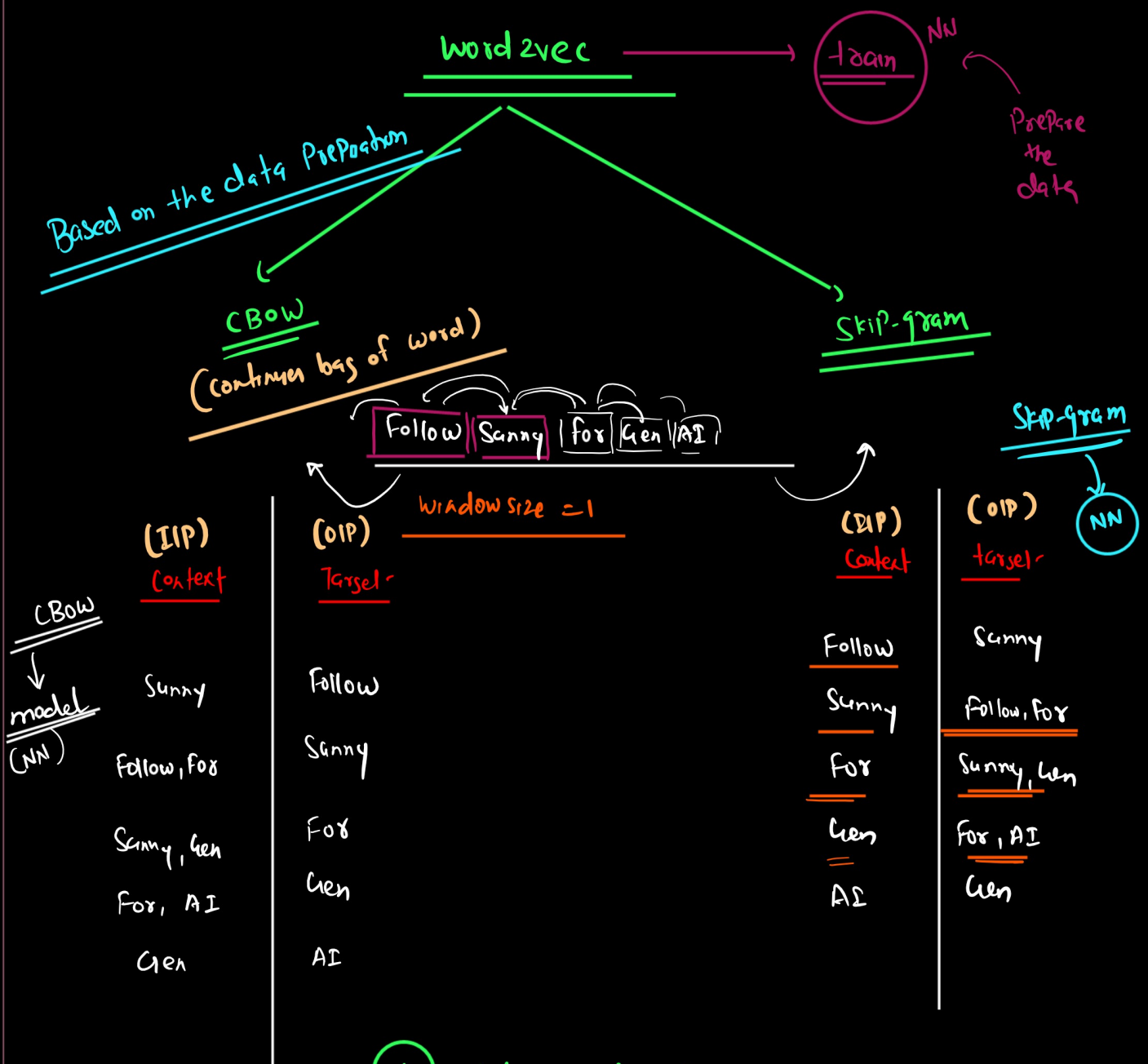
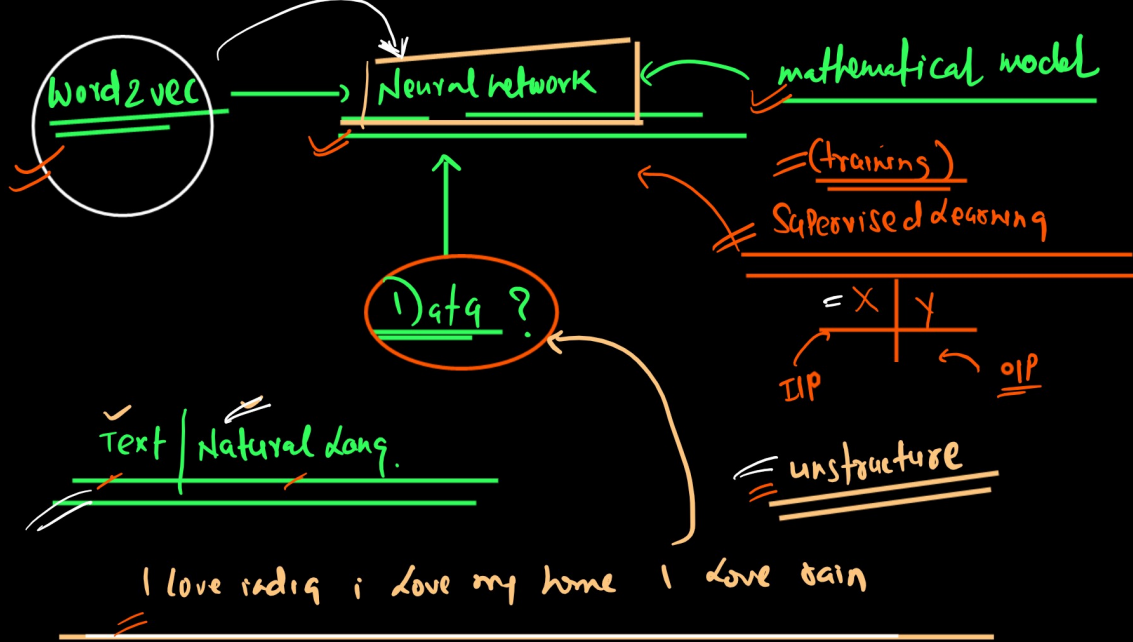
Window Size → hyperparameter (experiment)

5, 10, 15, 20, ...

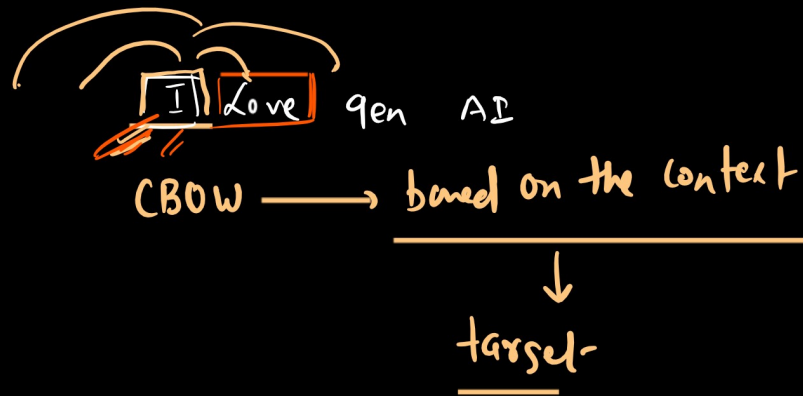
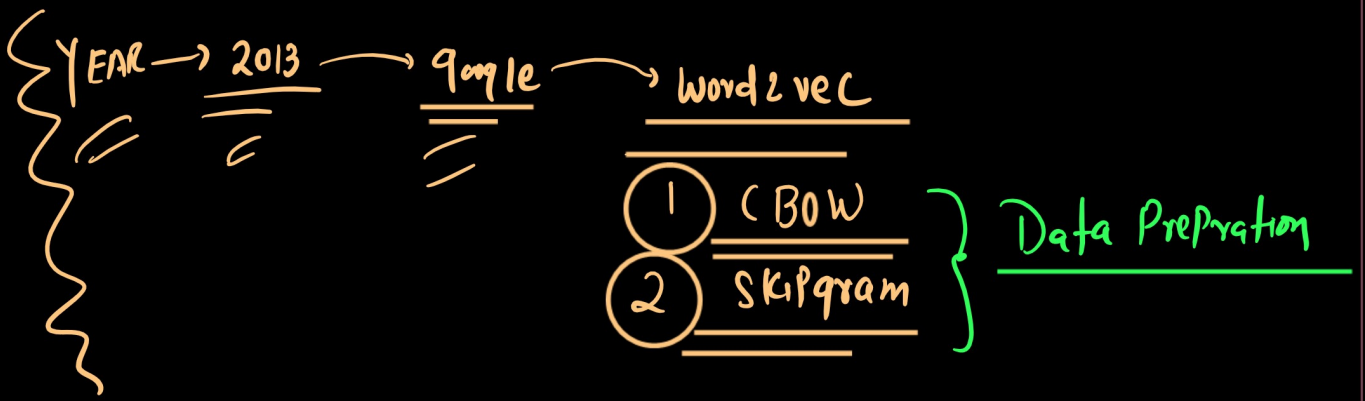
(Window size) ↑↑↑ Context ↑↑↑↑

very very high

Norse



- 1 What is word2vec?
- 2 CBOW & Skip-gram

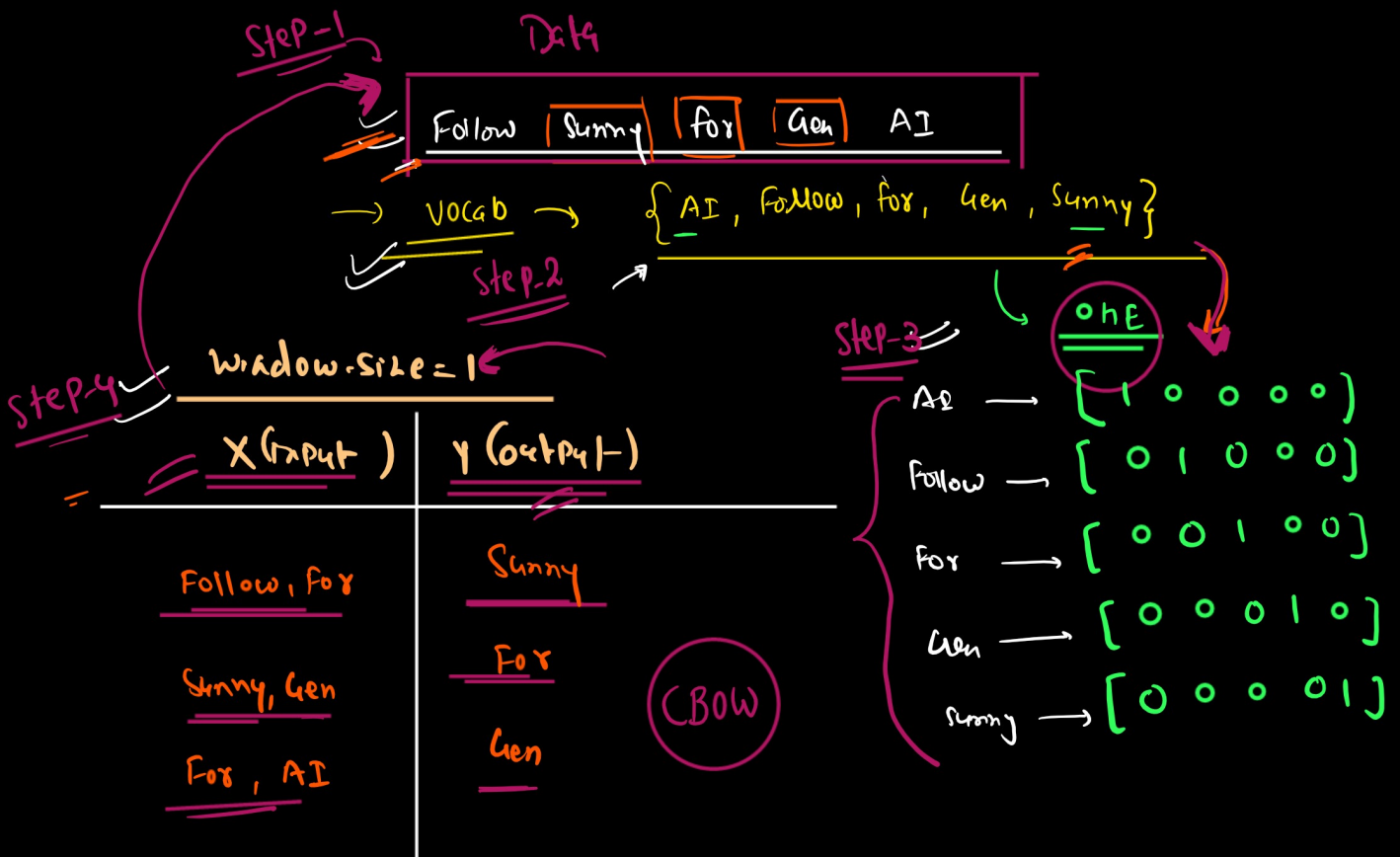


Skipgram →

Data Preparation

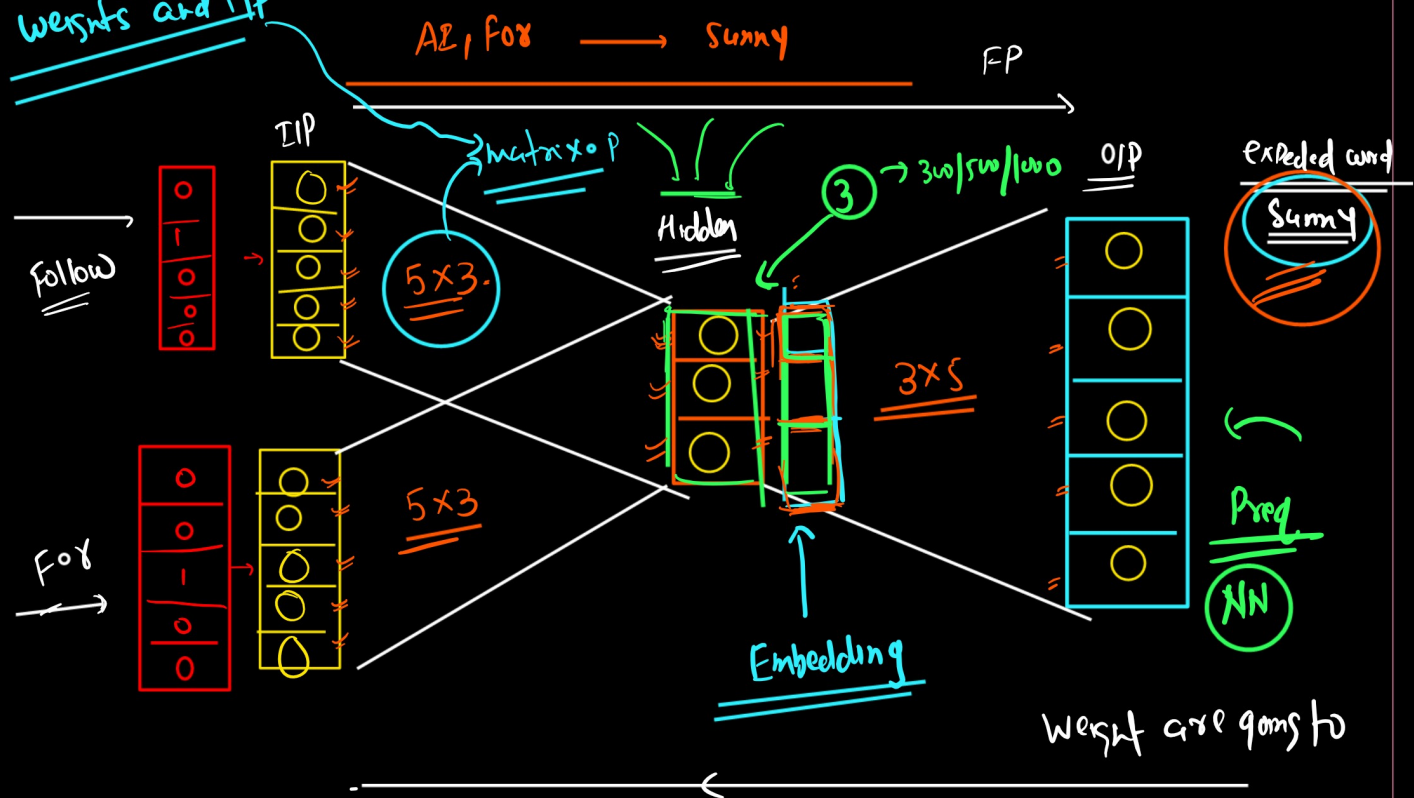
model being train

embedding



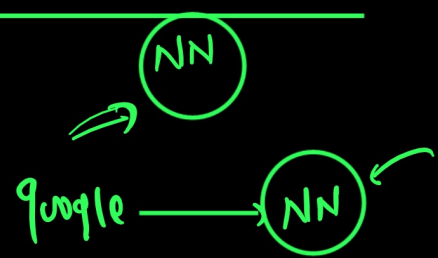
NN
training

weights and r/p



word → vector

be assigned
best possible weight.
(embedding vector)

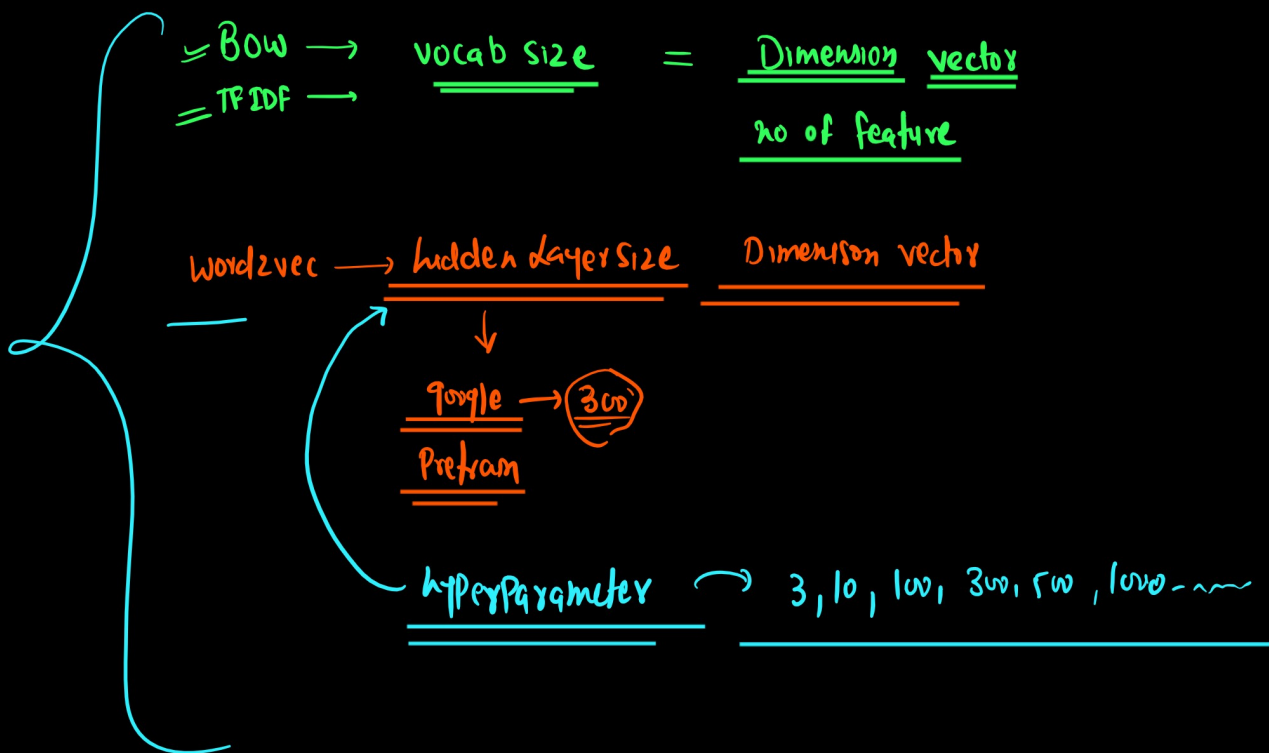
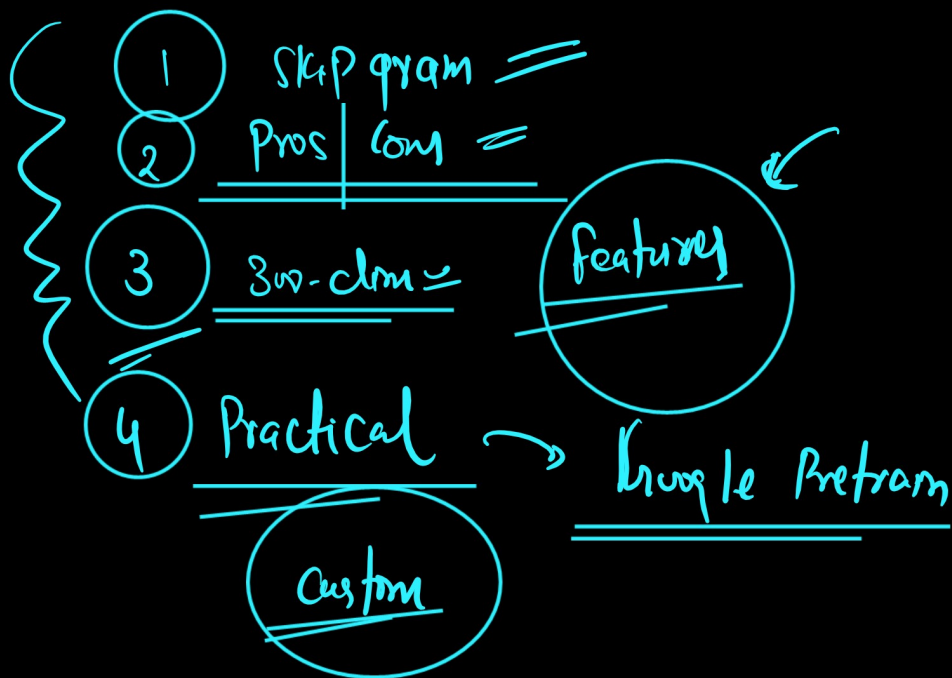


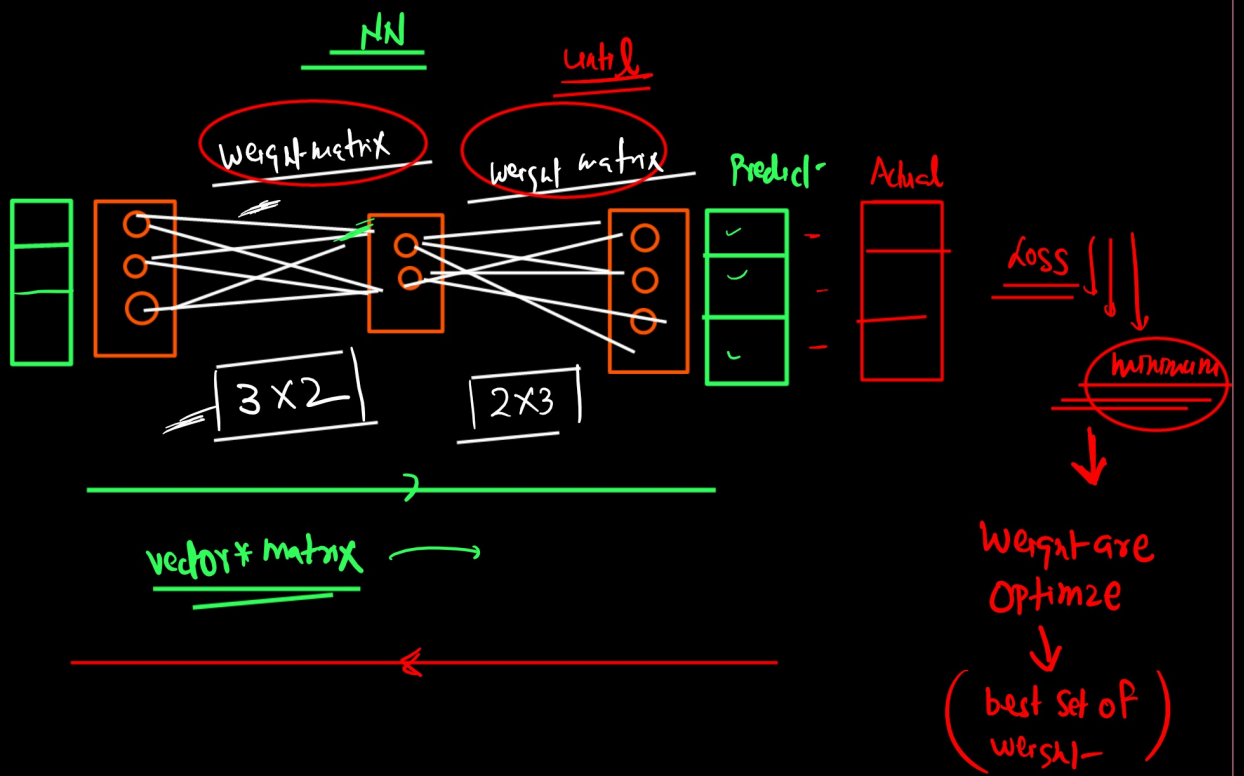
how hidden layer
1 hidden layer

with 3 neurons.

Simple architecture







Pros

1. Efficient & Fast Training

Shallow model (no deep layers)

2. Semantic Understanding

Similar meaning words → close vectors

Example:

king ≈ queen

car ≈ vehicle

Captures:

Similarity

Relationships

3. Vector Arithmetic

king - man + woman ≈ queen

4. Pretrained embeddings available

Google News vectors (300d)

Easy to reuse

5. General-purpose embeddings

Can be used in:

NLP pipelines

ML models

Clustering / search

layer

I/P hidden O/P

context

1-hidden layer

Neurons → hyperpara

3

300

any value

Example
Practically

very popular

2018 → NLP

bank → static & dynamically

Cons

1. Context-independent
Same word = same vector always
bank (river) = bank (finance)
2. Fixed embeddings
Vector would be static
will not change according to the sentence context

3. No handling of OOV (Out-of-Vocabulary)

New word?

"ChatGPT" → unknown

4. Cannot handle subwords

Words like:

running, runner, runs

Treated separately

No morphology understanding

Compared to:

BERT

GPT

Word2Vec:

No attention

No deep context

No dynamic embeddings

Google model

chatspr

2013

transformer

token

Diff

River bank

Finance bank

Same

Internally

embedding

word2vec

5 more

lot free

PAID SKIP

OpenAI

Google

HF

free

free

free

key

key

Live demo

Embedding model

text/image

prog. Openrouter LLM API